Multi-Agent Simulation Engine — operator behaviour & conjunction negotiation

SPEC-SW-006 v0.2 OVERVIEW Jeremie Nunez

1 Context

Thalamus answers *what is* and *what just happened* in the SSA knowledge graph. Fish answer a different question: *what would happen if this scenario were perturbed many times?* This spec documents the multi-agent simulation engine hosted in `packages/sweep`: a swarm creates isolated `sim_run` fish, runs them through short turn loops, and collapses terminal outcomes through app-owned aggregation / promotion paths.

Inspiration: MiroFish's five-stage pipeline (*graph build* → *environment setup* → *simulation* → *report* → *deep interaction*). This spec adapts that pattern to SSA, reusing the existing cortex registry, LLM transport, pgvector store, and BullMQ queues rather than running a Python sidecar.

Current implementation map

The current code is intentionally more generic than the original UC1/UC3 sketch:

- `SwarmService.launchSwarm` accepts `kind`, `title`, `baseSeed`, `perturbations`, and runtime config; the SSA pack validates supported kinds through `SsaKindGuard`.
- A fish is one `sim_run` under one `sim_swarm`; uniqueness is `(swarm_id, fish_index)`.
- Each fish owns its `sim_agent` rows, `sim_turn` timeline, and `sim_agent_memory`. Memory reads are scoped by `simRunId` and `agentId`; fish never share memory.
- Supported SSA kinds today are `uc1_operator_behavior`, `uc3_conjunction`, `uc_telemetry_inference`, and `uc_pc_estimator`.
- `uc3_conjunction` uses the sequential runner. The other kinds use the DAG runner; telemetry and PC estimator are single-turn flows by `driverForKind`.
- `AggregatorService` is not an LLM reporter. It clusters terminal observable summaries with embeddings when available, falls back to pack action bucketing, and exposes quorum / modal / divergence metrics. App-owned `SimOutcomeResolver` decides which aggregate becomes a snapshot, report, or suggestion.

Core idea — the swarm, not the sim

The primary unit of value is NOT a single high-fidelity simulation. It is a **swarm of cheap small-model "fish"** that collectively cover the space of plausible outcomes. Each fish is a short mini-sim with a perturbed seed (different god-event, different initial assumption, different persona tuning). The aggregator clusters outcomes and produces a distribution — "in 62% of futures, operator A maneuvers; in 28%, B maneuvers; in 10%, negotiation fails and escalates".

A single run is just a swarm of size 1. The determinism target is fixture / recorded replay under a fixed seed and perturbation set; live cloud mode is intentionally not treated as byte-stable.

Four SSA use-case slots are currently wired:

- **UC1 — Operator behaviour swarm.** N agents (one per operator), run for a short compressed horizon, across K fish ($K \approx 50$–100) each with a different god-event perturbation (regulation variants, ASAT event on different satellites, launch surges at different scales, debris cascade severities). Output: an aggregate distribution over operator decisions — "under these perturbations, what fraction of futures trigger a Starlink slot-share reshuffle?".

- **UC3 — Conjunction negotiation swarm.** A 2-agent swarm triggered from a conjunction context. K fish ($K \approx 30$–50) each perturb the seed (delta-v budget, risk tolerance, insurance-loss assumption). Output: a distribution over resolutions (who maneuvers, split ratio) plus a confidence band. The SSA outcome resolver may turn a strong modal resolution into a `sweep_suggestion`, annotated with the distribution for reviewer context.
- **Telemetry inference.** A single-subject, single-turn DAG fish estimating scalar telemetry fields under perturbed priors.
- **PC estimator.** A single-turn DAG fish estimating conjunction probability inputs for one conjunction event.

Fish turns call a cortex skill through `callNanoWithMode`. The execution knobs currently consumed at turn time are the global `sim.fish` runtime overrides (model, reasoning effort, output tokens, temperature) plus the global Thalamus transport mode. Launch-level `llmMode` / `nanoModel` are stored in swarm and run config for launcher contracts and reporting, but they are not a per-fish execution override in the current runner.

2 Goals

- **Swarm-first**: the orchestrator's primary API is `launchSwarm`; a single sim is $K = 1$. Each fish is a cheap nano-driven mini-sim with a perturbed seed.
- **Coverage over fidelity**: prefer 100 short cheap fish to 1 long expensive sim. The aggregator produces a distribution of outcomes, not a verdict.
- Model long-lived **agents within each fish** (not one-shot cortex calls) with stable identity, persona, and memory across turns.
- Drive **fish execution** over BullMQ with parallel fan-out. The `swarm-fish` worker defaults to concurrency 8 unless the process injects another worker concurrency; launch config stores `fishConcurrency` but does not currently reconfigure an already-running worker.
- Persist **per-agent memory** in pgvector scoped by `(sim_run_id, agent_id)` so fish cannot bleed into each other.
- Reuse existing LLM pathways: fish bodies use `callNanoWithMode` against the pack-selected cortex skill and validate JSON with `buildTurnResponseSchema`.
- Reuse the **cortex registry** as the per-turn reasoning primitive — no new LLM abstraction.
- Produce an aggregate snapshot through `AggregatorService` plus app-owned `SimOutcomeResolver`; domain promotion decides whether to emit Sweep suggestions.
- Wire supported sim kinds into Sweep so modal, quorum-backed outcomes can feed reviewer suggestions instead of bypassing HITL.

3 Non-Goals

Non-Goal

This spec does NOT re-implement orbital propagation, does NOT compute real delta-v budgets, and does NOT produce legally-actionable maneuver plans. All physics is heuristic: mass-based cost bands (already available via `queryReplacementCost`) and fleet-level proxies. Realism is narrative, not numerical.

Non-Goal

This spec does NOT introduce streaming LLM calls inside the sim loop. Each fish turn is a bounded `callNanoWithMode` invocation through the pack-selected cortex skill; streaming remains outside the sim runner path.

Non-Goal

This spec does NOT require MiroFish's Python runtime, Zep Cloud, or OASIS. The runtime surface is the existing Postgres `sim_*` schema plus the Sweep BullMQ workers / queues; there is no Python sidecar.

4 Invariants

Invariant

Agents are append-only within a fish. A `sim_agent.id` is scoped to one `sim_run`; `sim_agent_memory` rows are appended and scoped by `(sim_run_id, agent_id)`.

Invariant

A **turn** is the atomic unit of persisted sim progress. Sequential turns persist one actor turn plus memory rows together; DAG turns persist the successful actor turns plus cross-agent observations as one batch. Fish failure and retry policy is owned by the `swarm-fish` worker.

Invariant

God-view injections (policy, event) are themselves `sim_turn` rows with `actor_kind = 'god'` — they live in the same timeline as agent turns so the narrative report can interleave them.

Invariant

No cross-sim memory bleed. An agent bound to operator `X` in `sim_run` `A` cannot read memory rows written in `sim_run` `B`. Memory is scoped by `(agent_id, sim_run_id)`.

Invariant

UC3 negotiation swarms produce at most one `sweep_suggestion` per swarm (not per fish), tagged with `sim_swarm_id`. The suggestion payload carries the modal resolution plus the full outcome distribution as metadata. Individual fish never emit suggestions directly.

Invariant

A swarm should be deterministic under fixed seed / perturbation inputs when the actual nano transport layer is running from complete fixtures. Live `cloud` mode is evaluated statistically through the aggregate, not by byte equality.

Invariant

Fish within a swarm are *independent* — no fish reads another fish's memory or timeline. Cross-fish aggregation happens only in the aggregator, which reads `sim_turn` rows grouped by `sim_run_id`. This guarantees embarrassingly-parallel fan-out.

Invariant

Swarm aggregation is fail-soft: if f of K fish fail (LLM error, timeout), the aggregator proceeds when successful fish meet `quorumPct` (default 0.8). Below quorum, the swarm transitions to `failed` and no suggestion is emitted. `perFishTimeoutMs` is stored in launch config; the current inline fish loop does not enforce it directly.

Invariant

Thalamus never imports Sweep. The sim kernel lives in `packages/sweep/src/sim/`; SSA skills and domain prompts live in the app pack under `apps/console-api/src/agent/ssa/skills/`. Sweep consumes the public cortex registry and nano caller surfaces; app-specific skill names enter through ports.

Invariant

`sim_turn` is the canonical, mode-agnostic timeline. Both drivers (DAG and Sequential) write every agent action and every god event there. Memory, terminal views, aggregation, and scoped chat surfaces consume `sim_turn` / `sim_agent_memory` rather than transient per-turn KG rows, so the two modes are observationally aligned.

Invariant

Per-fish turns do not directly create reviewer suggestions. Turn actions stay in `sim_turn`; aggregate / outcome resolver / promotion code decides whether a modal result becomes a `sweep_suggestion`, report finding, or only an aggregate snapshot. Rationale: keep the KG signal-rich and keep noisy negotiation traces scoped to the sim run.

5 Architecture

5.1 Swarm layer (top)

The swarm is the public API. It fans out K fish over BullMQ, each fish is one `sim_run`, each fish picks a driver (DAG or Sequential) by its kind. The aggregator reads `sim_turn` across all fish in the swarm and emits an aggregate distribution / snapshot.

```
// swarm.service.ts
launchSwarm({
  kind: "uc1_operator_behavior" | "uc3_conjunction",
  baseSeed: SeedRefs,                    // fleet, finding, horizon
  perturbations: PerturbationSpec[],     // K entries
  config: { llmMode, quorumPct, perFishTimeoutMs, fishConcurrency,
     nanoModel, seed },
}): Promise<{ swarmId: number }>
```

Each `PerturbationSpec` is applied to the base seed to produce a fish-specific seed. Examples:

- `{ kind: "god_event", event: { kind: "asat_event", targetSatelliteId: 42 } }`
- `{ kind: "constraint_override", agentIndex: 0, overrides: { maxDeltaVBudgetPerSat: 50 } }`
- `{ kind: "persona_tweak", agentIndex: 0, riskProfile: "aggressive" }`
- `{ kind: "launch_surge", regimeId: 3, extraSatellites: 100 }`
- `{ kind: "delta_v_budget", agentIndex: 1, maxPerSat: 25 }`

- `{ kind: "pc_assumptions", hardBodyRadiusMeters: 20, covarianceScale: "loose" }`
- `{ kind: "noop" }` only when the launcher includes an explicit baseline perturbation.

5.2 Two drivers, one fish

The sim engine ships with **two turn drivers** selected by `sim_run.kind`:

- **DAG driver** (UC1 / telemetry / PC) — one turn = all in-scope agents build context, call `callNanoWithMode` through the selector/fallback skill path, then persist successful `sim_turn` rows plus self/observation memories as one batch. Optimal when N agents decide simultaneously.
- **Sequential driver** (UC3) — one turn = one speaker selected by `turnIndex % agents.length`. The runner calls the same selector/fallback skill path, writes one `sim_turn` plus memory rows, and marks the run done on terminal `accept` / `reject`. Optimal for tight negotiation back-and-forth.

Both drivers write to the same `sim_turn` table — the canonical, mode-agnostic timeline consumed by terminal views, memory, and aggregation. The concrete skill name is selected by the app pack through `SimCortexSelector` with the hints currently supplied by the runner; when those hints do not identify a telemetry or PC target, the SSA selector falls back to `sim_operator_agent`.

Promotion. Every agent action is written to `sim_turn`. Reviewer-visible suggestions and report findings are aggregate-level side effects owned by the app's outcome resolver and promotion service, not by individual fish turns.

5.3 Package layout

```
packages/sweep/src/sim/
  types.ts                      // SimConfig, SwarmConfig, turn/action
      types
  schema.ts                     // Zod turn-output helpers
  agent-builder.ts              // subject snapshot -> sim_agent +
      persona
  memory.service.ts             // fish-scoped memory read/write
  sim-orchestrator.service.ts   // create fish, seed god turns,
      standalone runs
  swarm.service.ts              // fan-out K fish, aggregate gate
  aggregator.service.ts         // cluster terminal outcomes, quorum/
      modal/divergence
  turn-runner-dag.ts            // parallel actor turns
  turn-runner-sequential.ts     // alternating negotiation turns
  turn-runner.utils.ts          // shared prompt/context/nano call path
  promote.ts                    // terminal/action helpers
  ports/                        // subject, persona, prompt, queue,
      store, resolver ports
  http/                         // HTTP adapters for service split

packages/sweep/src/jobs/workers/
  sim-turn.worker.ts            // standalone turn worker
  swarm-fish.worker.ts          // one fish lifecycle, drains turns
      inline
  swarm-aggregate.worker.ts     // outcome resolver + aggregate snapshot
       / close

apps/console-api/src/agent/ssa/sim/
  kind-guard.ts                 // SSA supported kinds + driver
      selection
```

```
  prompt-renderer.ts              // domain prompt composer
  cortex-selector.ts              // pick cortex skill per sim kind
  aggregation-strategy.ts         // labels + fallback clustering
  promotion.ts                    // app-owned suggestion / report
     promotion
  aggregators/                    // telemetry and PC aggregate
     specializations
```

5.4 Data model

Five tables in `packages/db-schema/src/schema/sim.ts`. Abbreviated to the fields that define the architecture:

```
sim_swarm:
  id, kind, title, base_seed, perturbations, size, config,
  status, outcome_report_finding_id, suggestion_id

sim_run:
  id, swarm_id, fish_index, kind, seed_applied, perturbation,
  config, status, report_finding_id
  unique(swarm_id, fish_index)

sim_agent:
  id, sim_run_id, operator_id, agent_index, persona,
  goals, constraints
  unique(sim_run_id, agent_index)

sim_turn:
  id, sim_run_id, turn_index, actor_kind, agent_id,
  action, rationale, observable_summary, llm_cost_usd
  unique(sim_run_id, turn_index, agent_id) where agent_id is not null

sim_agent_memory:
  id, sim_run_id, agent_id, turn_index, kind, content,
  embedding, retired_at
  index(sim_run_id, agent_id)
```

The schema uses bigint IDs and a follow-on HNSW migration for `sim_agent_memory.embedding`. The operational invariant is the scoped B-tree index first: every memory lookup starts from `(sim_run_id, agent_id)`, then uses vector similarity or recency inside that fish-local scope.

5.5 Turn loop — DAG driver (UC1)

```
// turn-runner-dag.ts --- one turn = all agents act in parallel
async function runTurnDag(simRunId: number, turnIndex: number) {
  const agents = await loadTurnRunnerAgents(store, simRunId);
  const contexts = await Promise.all(
    agents.map((agent) => buildContext({ simRunId, agent, turnIndex })),
  );

  const llmResults = await Promise.allSettled(
    contexts.map((ctx) => callTurnAgent({ ctx, deps, logger })),
  );

  const successes = collectSuccessfulAgentResponses(agents, llmResults);
  await store.persistTurnBatch({
    agentTurns: successes.map(toSimTurn),
    memoryRows: buildSelfAndObservationMemoryRows(successes, agents),
```

```
  });
}
```

5.6 Turn loop — Sequential driver (UC3)

```
// turn-runner-sequential.ts --- one turn = one agent speaks
async function runTurnSequential(simRunId: number, turnIndex: number) {
  const agents = await loadTurnRunnerAgents(store, simRunId);
  const speaker = agents[turnIndex % agents.length];
  const ctx = await buildContext({ simRunId, agent: speaker, turnIndex
      });
  const response = await callTurnAgent({ ctx, deps, logger });

  await store.persistTurnBatch({
    agentTurns: [toSimTurn(speaker, response)],
    memoryRows: [
      toSelfMemory(speaker, response),
      ...toObservationMemories(agents, speaker, response),
    ],
  });

  if (isTerminal(response.action)) {
    await store.updateRunStatus(simRunId, "done", new Date());
  }
}
```

5.7 God channel

God events are injected through the console-api sim control routes, primarily `POST /api/sim/runs/:id/inject`; kernel routes also expose `/api/sim/runs/:id/god-turns` and `/api/sim/runs/:id/god-events`. Payload is stored as a `sim_turn` row with `actor_kind = 'god'`. Agents see god events as part of their observable timeline — no special branch in the agent prompt.

5.8 UC3 wiring

UC3 is launched through the app's sim launcher surfaces rather than a hidden Sweep kernel shortcut. The SSA pack validates `uc3_conjunction` with exactly two `subjectIds`, runs the sequential driver for up to 20 turns, aggregates terminal outcomes, then lets `SsaSimOutcomeResolverService` decide whether a modal `accept` / `maneuver` outcome becomes a Sweep suggestion. Reviewer handling remains the normal Sweep inbox flow.

6 Interface

```
// kernel-facing service surface
SwarmService.launchSwarm({
  kind,
  title,
  baseSeed,
  perturbations,
  config,
  createdBy,
})

SimOrchestrator.startStandalone({
  kind,
```

```
  title,
  subjectIds,
  llmMode,
  nanoModel?,
  seed?,
})

// console-api routes
POST  /api/sim/telemetry/start
POST  /api/sim/pc/start
POST  /api/sim/standalone/start

POST  /api/sim/swarms
GET   /api/sim/swarms/:id/status
GET   /api/sim/swarms/:id/terminals
PATCH /api/sim/swarms/:id/aggregate
POST  /api/sim/swarms/:id/close

GET   /api/sim/runs/:id
POST  /api/sim/runs/:id/turns/batch
POST  /api/sim/runs/:id/inject
GET   /api/sim/runs/:id/memory/recent
POST  /api/sim/runs/:id/memory/search
```

7 Behavioral scenarios

The scenarios below describe the current sim-kernel behavior and avoid route names except where the route is part of the public console-api surface.

Scenario: Swarm launch creates isolated fish

Given `SwarmService.launchSwarm` receives K perturbations and a supported SSA `kind`
When the launch succeeds
Then one `sim_swarm` row is created with `size = K`
And K `sim_run` rows exist with unique `(swarm_id, fish_index)`
And every fish receives its own `sim_agent` rows and is enqueued as one `swarm-fish` job.

Scenario: UC3 sequential negotiation

Given an SSA `uc3_conjunction` swarm with exactly two `subjectIds`
When `swarm-fish.worker` drains one fish
Then the `SsaKindGuard` selects the sequential driver
And each turn selects one speaker by `turnIndex % agents.length`
And terminal `accept` or `reject` marks the fish `done` before the 20-turn cap.

Scenario: DAG-style parallel actor turn

Given an SSA `uc1_operator_behavior` fish with N agents
When the DAG runner executes turn t
Then it builds N contexts, calls the selected cortex skill in parallel, and persists successful `sim_turn` rows plus self / observation memory rows in one batch.

Scenario: God-view event injection

Given a fish is running
When the operator posts `/api/sim/runs/:id/inject`
Then a `sim_turn` row with `actor_kind = "god"` is appended
And later agent contexts include that event through the observable timeline.

Scenario: Memory scoping

Given operator A participated in sim run X (20 turns of memory) and sim run Y (5 turns)
When agent-A in run Y queries its memory at turn 3
Then the vector search returns only rows where `sim_run_id = Y`
And zero rows from run X are returned even if semantically similar.

8 Acceptance criteria

Acceptance Criterion AC-1 Schema

Schema validation expects `sim_swarm`, `sim_run`, `sim_agent`, `sim_turn`, and `sim_agent_memory` to exist, with `sim_run` unique on `(swarm_id, fish_index)` and memory indexed by `(sim_run_id, agent_id)`.

Acceptance Criterion AC-2 Kind guard

The SSA app pack rejects unsupported sim kinds, requires exactly two `subjectIds` for `uc3_conjunction`, one `subjectId` plus `telemetryTargetSatelliteId` for telemetry inference, and `pcEstimatorTarget` for PC estimator swarms.

Acceptance Criterion AC-3 Aggregation

`AggregatorService` returns `quorumMet=false` with no modal outcome when successful fish do not meet quorum; when quorum is met, it emits clusters, modal action, and `divergenceScore`.

Acceptance Criterion AC-4 God injection

`POST /api/sim/runs/:id/inject` appends a god turn that is visible to later agent contexts through `MemoryService.recentObservable` / god-event loading.

Acceptance Criterion AC-5 Resolution boundary

No individual fish turn emits a reviewer suggestion. Suggestions come from app-owned aggregate / promotion paths, then pass through the same Sweep reviewer and resolution flow as audit suggestions.

Acceptance Criterion AC-6 Isolation

Thalamus package continues to typecheck and pass its existing tests with zero imports from `@interview/sweep`.

9 Current follow-ups

1. Keep `observableSummary` in the same validated turn response unless leakage or evaluation data proves a second pass is worth the cost.
2. Add explicit docs/tests around large UC1 swarms: the current DAG driver is parallel per turn, but wall-clock and queue pressure depend on `fishConcurrency`, agent count, and model latency.
3. Document no-agreement UC3 outcomes as aggregate snapshots first; only promote to suggestions when the outcome resolver has a reviewer-useful modal action.

10 References

- MiroFish README — https://github.com/666ghj/MiroFish/blob/main/README.md
- SPEC-SW-001 Nano Sweep (`nano-sweep.tex`)
- SPEC-SW-002 Finding Routing (`finding-routing.tex`) — audit-to-suggestion routing background
- SPEC-SW-003 Resolution (`resolution.tex`) — consumes accepted suggestions
- `packages/sweep/src/sim/turn-runner.utils.ts` — current `callNanoWithMode` per-turn primitive